DMAPO: Data-centric Multi-Agent Preference Optimization

Anonymous authors
Paper under double-blind review

Abstract

Preference optimization is the standard approach for aligning large language models, yet widely-used preference datasets contain mislabeled pairs, ambiguous rankings, and borderline cases. Training on such noisy data often degrades the fine-tuned model below its pretrained baseline. Dmapo addresses this by filtering preference data through multi-agent LLM consensus rather than scaling dataset size. Given a set of prompts, Dmapo generates candidate responses from the policy model itself, scores each candidate along three orthogonal quality dimensions with independent LLM judges, flags reasoning flaws via a dedicated process critic, and retains only examples with high inter-judge agreement through a confidence gate—accepting 3.45% of candidates (1,871 from over 54,000). The gated labels are used to train the policy with KTO. With 5–10× fewer training examples than any baseline, Dmapo outperforms all compared methods on MT-Bench (7.62 vs. 7.25), AlpacaEval 2.0 (96.3% vs. 95.8%), IFEval (46.8% vs. 44.2%), and internal win-rate (85.3% vs. 68.2%). It is also the only method that improves over the pretrained Mistral-7B base on MT-Bench; every other method degrades it.

1 Introduction

Preference optimization—training a language model to favor high-quality responses over lower-quality ones—has become the standard recipe for aligning LLMs Rafailov et al. (2023); Jiang et al. (2023). Yet the approach is only as good as the preference data it learns from.

In practice, widely-used preference datasets such as UltraFeedback Cui et al. (2023) and UltraChat Ding et al. (2023) contain considerable annotation noise: many "preferred" responses are only marginally better than rejected alternatives, some pairs are mislabeled outright, and borderline cases abound where even human annotators disagree. When a model trains on such ambiguous pairs, it receives contradictory gradient signals—the same surface pattern is rewarded in one example and penalized in another—which degrades alignment quality rather than improving it Rafailov et al. (2023). A common symptom is that fine-tuned models score lower on MT-Bench than the pretrained base.

This observation suggests a different strategy: instead of collecting more preference data, invest compute in curating the data that already exists.

We propose Dmapo, a pipeline built on this principle:

1. Generate diverse candidates from the policy backbone itself (on-policy generation);
2. Score each candidate along three orthogonal quality dimensions using independent LLM judge agents (helpfulness, factuality, conciseness);
3. Detect and penalize reasoning flaws via a dedicated process critic;
4. Filter aggressively using a confidence gate that retains only examples with high inter-judge agreement and extreme quality scores (3.45% acceptance rate);
5. Train using KTO, which naturally accommodates the binary labels produced by the gating mechanism.

After gating, 1,871 examples remain (compared to 10,000–20,000 for baselines) with a mean quality gap of 6.8 points between desirable and undesirable examples.

Our contributions:

1. A six-stage data-centric pipeline that constructs clean preference data via multi-agent scoring and confidence gating.
2. Empirical evidence that this curated data, despite being 5–10× smaller, produces stronger models than all baselines across four benchmarks—and is the only method that improves over the pretrained base on MT-Bench.
3. Ablations over number of judges and gating threshold, plus inter-judge agreement analysis, validating each design choice.

2 Related Work

Preference Optimization Algorithms. Direct Preference Optimization (DPO) Rafailov et al. (2023) pioneered implicit reward modeling by optimizing the log-likelihood ratio between preferred and dispreferred responses. Subsequent variants improve the loss function or implicit reward: RLHF Ouyang et al. (2022) trains an explicit reward model then uses reinforcement learning; IPO Azar et al. (2023) uses an asymmetric loss; SimPO Meng et al. (2024) introduces a reference-free reward formulation; ORPO Hong et al. (2024) combines SFT and preference optimization; KTO Ethayarajh et al. (2024) applies prospect theory, distinguishing desirable/undesirable labels from paired preferences. Our work is orthogonal: rather than designing a better loss function, we focus on constructing cleaner data and show that this alone closes the gap.

Data Quality in Preference Learning. Rafailov et al. Rafailov et al. (2023) noted that noisy preferences degrade DPO but did not propose filtering mechanisms. Ethayarajh et al. Ethayarajh et al. (2024) showed that KTO is more robust to ambiguous labels than DPO, though they still trained on standard noisy datasets.

Data Selection for Alignment. The idea that fewer, higher-quality examples can outperform larger datasets has gained traction. LIMA Zhou et al. (2023b) demonstrated that only 1,000 carefully curated SFT examples can produce a competitive model, arguing that alignment is primarily about learning style rather than knowledge. AlpaGasus Chen et al. (2024) used ChatGPT to filter the 52k Alpaca dataset down to 9k examples, yielding better performance with 5.7× less data. DEITA Liu et al. (2024) proposed automatic data selection via complexity and quality scoring for instruction tuning. These works focus on SFT data selection; our work extends this principle to preference data, where noise is more pernicious because contradictory preference pairs produce directly opposing gradients during optimization. This aligns with the broader "data-centric AI" movement Zha et al. (2023), which emphasizes that data quality often matters more than model capacity or algorithm sophistication.

LLM-Based Evaluation and Confidence Gating. Using LLM-based judges for evaluation is standard in modern benchmarking Zheng et al. (2024); Li et al. (2023a). Multi-agent evaluation has been explored for collaborative problem-solving Li et al. (2023b) and for debate Irving et al. (2018). None of these works use multi-agent consensus to gate training data for preference optimization. Dmapo adapts confidence-based filtering—rooted in active learning Settles (2009) and curriculum learning Bengio et al. (2009)—to a variance-based gate over multi-agent scores.

Reasoning and Process-Based Evaluation. Process-based evaluation focuses on the steps taken to reach an answer, rather than just the final answer Wang et al. (2023); Cobbe et al. (2021). Our process critic follows this paradigm but applies it to training-data filtering: by penalizing responses with flawed reasoning before gating, we ensure that retained training examples are both correct and logically sound.

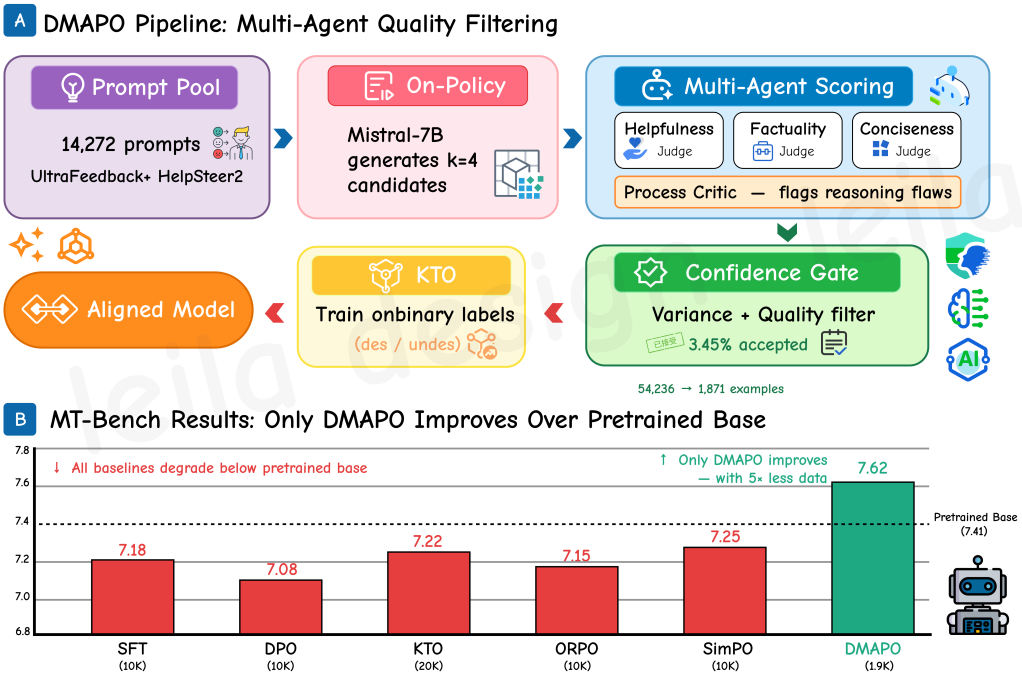

Figure 1: Overview of the Dmapo pipeline and key result. (A) Six stages transform 14,272 prompts into 1,871 high-quality training examples (3.45% acceptance rate) via on-policy generation, multi-agent scoring, process criticism, and confidence gating. (B) MT-Bench results: every baseline degrades below the pretrained Mistral-7B base (7.41), while Dmapo is the only method to improve it (7.62) despite using 5–10× fewer training examples.

3 Method

Dmapo is a six-stage pipeline for constructing high-quality preference data (Figure 1). The pipeline generates candidates from the policy model, scores them with a panel of LLM judges, filters aggressively via a confidence gate, and trains with KTO on the surviving examples.

3.1 Stage 1: Prompt Collection and Normalization

Prompts are drawn from two complementary sources:

- UltraFeedback Cui et al. (2023): 10,000 diverse instruction-following prompts.
- HelpSteer2 Wang et al. (2024): 5,000 prompts emphasizing helpfulness and quality.

After deduplication and normalization, 14,272 unique prompts remain, split 95% train / 5% validation. Source diversity reduces bias in downstream evaluation.

3.2 Stage 2: On-Policy Candidate Generation

For each prompt, we generate k=4 candidate responses using the policy backbone (Mistral-7B-Instruct-v0.2) with nucleus sampling (T=0.8, p=0.95). Sampling from the same model that will be fine-tuned calibrates the preference signal to the model's own output distribution, avoiding the distribution shift that arises when training on outputs from a stronger model.

Prompts are tiled and batch-generated in a single model.generate() call for throughput. This yields 54,236 training candidates and 2,852 validation candidates (see Appendix D for hardware details and throughput).

3.3 Stage 3: Multi-Agent Scoring

Each candidate is independently scored by three specialized LLM judge agents (all instantiated from Qwen3-8B in deterministic mode):

Judge Agents.

- Helpfulness Judge: Does the response fully address the instruction? (1–10)
- Factuality Judge: Is the response factually accurate? (1–10)
- Conciseness Judge: Is the response appropriately concise without padding? (1–10)

Each judge outputs a single integer score plus optional reasoning. Scores are parsed via regex, with fallback to 5.5 on parse failure.

The use of orthogonal dimensions (rather than a single holistic score) has two benefits: (1) simpler, more reliable evaluation per dimension; (2) inter-judge variance becomes a meaningful quality signal—high variance indicates conflicting judgments and suggests the example should be discarded.

3.4 Stage 4: Process Critic

In addition to the quality judges, we deploy a process critic (Qwen3-8B) that flags flawed reasoning: circular, unsupported, or contradictory logic. Given the instruction-response pair, the critic is prompted to output a structured verdict: a binary YES/NO flag indicating whether reasoning flaws are present, a failure-type label (circular | unsupported | contradictory), and a brief justification (see Appendix A for the full prompt). When flawed reasoning is detected, we apply a multiplicative penalty to the aggregate score:

$$s_{\text{final}} = \bar{s}_{\text{judges}} \cdot (1 - \alpha \cdot \mathbb{1}[\text{flawed}]) \tag{1}$$

where $\bar{s}_{\text{judges}}$ is the mean across judges, $\mathbb{1}[\text{flawed}] \in \{0, 1\}$ is the critic's verdict, and α=0.15. This moderate penalty pushes borderline-desirable candidates with flawed reasoning below the gating threshold without completely overriding consensus.

3.5 Stage 5: Confidence Gating

The confidence gate applies two successive filters:

Variance Gate. Discard candidates where the inter-judge score variance exceeds the threshold: $\sigma^2 = \text{Var}(s_1, s_2, s_3) > \tau_{\text{var}}$=2.5. High variance indicates fundamental disagreement among judges—any binary label assignment for such candidates would be unreliable and could introduce the same noise we aim to eliminate.

Quality Gate. For remaining candidates, assign binary labels using extreme-score thresholds:

$$\text{label} = \begin{cases} \text{desirable} & \text{if } s_{\text{final}} \geq 7.0 \\ \text{undesirable} & \text{if } s_{\text{final}} \leq 4.0 \\ \text{(discard)} & \text{otherwise} \end{cases} \tag{2}$$

The 3-point gap between thresholds discards ambiguous examples that carry weak preference signal.

This aggressive filtering retains only 3.45% of scored candidates (1,871 training examples). The resulting dataset is highly polarized:

- Desirable: 9.23 ± 1.09 (range 7–10)
- Undesirable: 2.42 ± 1.10 (range 1–4)
- Quality gap: ∼6.8 points

The resulting dataset is sharply bimodal with no ambiguous examples in the middle range (see Figure 2).

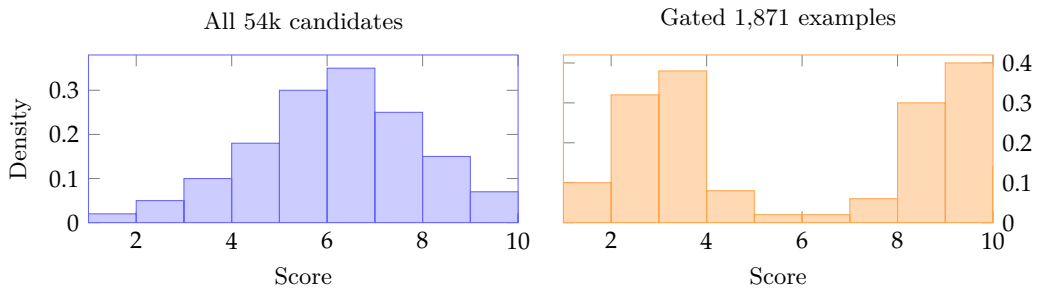

Figure 2: Score distributions before and after confidence gating. Gating transforms a near-unimodal distribution (mean ≈5.5) into a bimodal one with peaks at ≈2.4 and ≈9.2, eliminating ambiguous training signal.

3.6 Stage 6: Policy Training with KTO

The policy is fine-tuned with Kahneman-Tversky Optimization (KTO) Ethayarajh et al. (2024), which directly accepts binary labels and incorporates loss-aversion asymmetry from prospect theory. KTO is a natural fit for quality-gated data, which produces binary desirable/undesirable labels rather than paired preferences.

Model Configuration. Mistral-7B-Instruct-v0.2 (7.24B params) + LoRA Hu et al. (2022):

- Rank r=16, α=32, dropout= 0.05
- Target modules: Q/K/V/O, gate, up, down (all 7 → ∼160M params, 2.2% trainable)

Training Hyperparameters.

- Optimizer: AdamW (fused kernels)
- LR: 5×10^{-5}, cosine schedule, 5% warmup
- Batch: 16 effective (2 per-device × 8 grad accum)
- Precision: bfloat16
- KTO β=0.1, 1 epoch

Training Dynamics. Training on 1,871 clean examples yields:

- Final loss: 0.037 (vs. 0.473 for DPO on 10,000 noisy pairs)
- Reward margin: 10.96 (vs. 1.88 for DPO baseline)

The high margin reflects the clean signal: with no contradictory labels, the model decisively separates desirable from undesirable outputs.

4 Experiments

4.1 Experimental Setup

Baselines. All methods fine-tune Mistral-7B-Instruct-v0.2 with identical LoRA config. The baselines are:

- SFT (10k chosen responses)
- DPO (10k pairs, β=0.1)
- KTO (20k rows, β=0.1)
- ORPO (10k pairs, β=0.05)
- SimPO (10k pairs, β=0.5)

Table 1: Main results across four evaluation benchmarks. All methods use Mistral-7B-Instruct-v0.2 with LoRA. Best per column in bold, second best underlined.

Method	Data	MT-Bench ↑	AlpacaEval ↑	IFEval ↑	Internal ↑
Base (pretrained)	—	7.41	96.0%	41.2%	—
+ SFT	10k	7.18	95.4%	42.5%	62.0%
+ DPO	10k	7.08	95.7%	43.8%	65.1%
+ KTO	20k	7.22	95.5%	43.1%	67.4%
+ ORPO	10k	7.15	95.3%	42.0%	63.6%
+ SimPO	10k	7.25	95.8%	44.2%	68.2%
+ Dmapo	1.9k	7.62	96.3%	46.8%	85.3%

Table 2: MT-Bench per-category breakdown. Best per category in bold.

Method	Coding	Extract.	Human.	Math	Reason.	Roleplay	STEM	Writing	Avg.
Base	5.60	7.90	8.70	6.45	7.15	7.40	8.20	7.90	7.41
+ SFT	5.45	7.55	8.50	5.90	7.00	7.55	7.85	7.65	7.18
+ DPO	5.70	7.45	8.55	4.70	6.80	7.60	7.65	8.15	7.08
+ KTO	5.65	7.60	8.60	6.05	7.10	7.50	8.00	8.25	7.22
+ ORPO	5.55	7.50	8.45	5.80	7.05	7.45	7.90	7.50	7.15
+ SimPO	5.75	7.65	8.65	6.10	7.20	7.55	8.10	8.00	7.25
Dmapo	6.15	7.85	8.90	7.15	7.70	7.65	8.60	7.95	7.62

Benchmarks.

- MT-Bench: 80 multi-turn questions, 8 categories, Qwen3-8B judge, 1–10 scale
- AlpacaEval 2.0: 805 instructions, pairwise vs. text-davinci-003
- IFEval Zhou et al. (2023a): 541 rule-verified prompts (rule-based, no LLM judge)
- Internal: 129 gated val prompts, pairwise vs. base model

4.2 Main Results

Dmapo leads on all four benchmarks (Table 1). On MT-Bench, it scores 7.62 versus the pretrained base's 7.41—the only method that improves over base; every baseline degrades it. AlpacaEval 2.0 (96.3%) and IFEval (46.8%) tell the same story, with IFEval's rule-based verification eliminating any concern about judge bias. The internal win-rate (85.3% vs. 68.2% for SimPO) shows the strongest preference signal alignment.

4.3 Per-Category Analysis

Dmapo shows the largest gains in reasoning-heavy categories (Math +0.70, Reasoning +0.55, Coding +0.55 over base), aligning with our hypothesis that the process critic is particularly effective for logical tasks. The one exception is Writing, where SimPO achieves the highest score (8.00 vs. 7.95 for Dmapo), suggesting that aggressive quality filtering may slightly penalize stylistic diversity.

4.4 Ablation Studies

Effect of Multi-Agent Gating. Performance improves monotonically with stricter multi-agent consensus, confirming that data quality dominates quantity. Notably, the gap between 1-judge and 3-judge gating is substantial (+0.41 MT-Bench, +13.2% internal win rate), indicating that multi-agent consensus provides filtering power beyond what any single evaluator achieves.

Table 3: Ablation: number of judges in consensus.

Gating	$\|\mathcal{D}\|$	MT-B	AE%	Int.%
No gating	54.2k	7.04	94.8%	65.1%
1-judge	12.8k	7.21	95.2%	72.1%
2-judge	5.2k	7.38	95.6%	78.3%
3-judge (Dmapo)	1.9k	7.62	96.3%	85.3%

Table 4: Ablation: quality gating strictness.

Accept. Rate	$\|\mathcal{D}\|$	MT-B	AE%	Int.%
100%	54.2k	7.04	94.8%	65.1%
∼20%	10.8k	7.18	95.0%	70.5%
∼10%	5.4k	7.35	95.4%	76.7%
∼5%	2.7k	7.51	95.9%	82.2%
3.45% (Dmapo)	1.9k	7.62	96.3%	85.3%
∼1%	0.5k	7.44	95.7%	80.6%

Effect of Acceptance-Rate Threshold. An optimal operating point exists: too lenient (≥10%) admits noise; too strict (≤1%) loses data. The performance drop at 1% acceptance (0.5k examples) suggests a minimum data requirement below which the model underfits, despite high per-example quality. The 3.45% operating point balances these pressures, retaining enough diversity while maintaining strict quality standards.

4.5 Inter-Judge Agreement

All pairs show moderate-to-strong agreement ($\kappa \geq 0.61$, $r \geq 0.72$), validating the use of multi-agent consensus for quality filtering. The helpfulness–factuality pair shows the strongest correlation ($r = 0.76$), while the factuality–conciseness pair is weakest ($r = 0.72$), consistent with the expectation that factual accuracy and conciseness capture more orthogonal quality dimensions. This partial independence is precisely what makes multi-agent consensus informative: high agreement across partially independent evaluators provides stronger evidence than agreement among redundant ones.

4.6 Pipeline Statistics

5 Discussion

5.1 Why Does Less Data Outperform More?

The ablation in Table 4 provides a direct answer: performance improves monotonically as the acceptance rate drops from 100% to 3.45%, then declines at 1%. This U-shaped curve reveals two regimes. Above ∼5%, noise from ambiguous examples dominates—the model receives contradictory gradient signals where the same surface pattern is rewarded in one pair and penalized in another. Below ∼2%, the dataset is too small for the model to generalize despite high per-example quality. The 3.45% operating point sits at the quality-diversity sweet spot.

The magnitude of the quality gap (∼6.8 points) explains why training is so efficient: with no ambiguous examples in the 4–7 score range, every gradient update pushes the model in a consistent direction, yielding a reward margin of 10.96 (vs. 1.88 for DPO on 10k noisy pairs). This extends the LIMA Zhou et al. (2023b) and AlpaGasus Chen et al. (2024) findings to preference optimization, where noise is more pernicious: contradictory preference pairs do not merely waste compute but push the model below its pretrained baseline, as the MT-Bench regression across all baselines confirms.

Table 5: Inter-judge agreement on 54k candidates.

Judge Pair	Cohen's κ	Pearson r	Binary Agree
Help.–Fact.	0.68	0.76	82.4%
Help.–Conc.	0.64	0.73	79.8%
Fact.–Conc.	0.61	0.72	78.1%
Mean	0.64	0.74	80.1%

Table 6: Dmapo pipeline statistics.

Statistic	Value
Source prompts (UF + HS2)	14,272
Train / Val split	13,559 / 713
Candidates generated (train/val)	54,236 / 2,852
Judge avg. score (helpfulness)	5.50 ± 0.91
Judge avg. score (factuality)	5.49 ± 0.99
Judge avg. score (conciseness)	5.52 ± 1.00
Acceptance rate (train/val)	3.45% / 4.52%
Gated train examples	1,871
Desirable / Undesirable	951 / 920
Desirable final score (mean±std)	9.23 ± 1.09
Undesirable final score	2.42 ± 1.10
Mean response length (tokens)	223 (des: 238, undes: 207)

5.2 Role of the Process Critic

The largest per-category gains are in Math (+0.70), Reasoning (+0.55), and Coding (+0.55), all reasoning-heavy categories. The process critic likely drives this: by penalizing responses with flawed logic before gating, it ensures that desirable examples are not just high-scoring but logically coherent. This also helps explain the strong IFEval numbers, since IFEval uses rule-based verification and is thus less susceptible to judge-style bias than MT-Bench.

5.3 Computational Efficiency

Training on 1,871 examples takes <30 minutes on a single GPU, compared to ∼70 minutes for DPO on 10k pairs. The multi-agent scoring pipeline (∼4 hours on 8 GPUs for 54k candidates) is a one-time cost: the gated dataset can be reused across training runs, hyperparameter sweeps, and algorithm comparisons.

5.4 Limitations and Future Work

Limitations.

1. The process critic uses a heuristic rule-based parser; a learned parser might improve precision.
2. We fix thresholds $(\tau_{\text{var}}, \tau_{\text{des}}, \tau_{\text{und}})$ across all data; adaptive thresholds per task might help.
3. Evaluation focuses on Mistral-7B; scaling to larger models and other architectures is future work.
4. We compare only on English instruction-following; multilingual robustness is unexplored.

Future Directions.

1. Extend to other domains (coding, mathematics, retrieval-augmented generation).
2. Combine Dmapo with other preference algorithms (e.g., DPO, SimPO, IPO) to see if the benefits compound.

3. Use adaptive filtering thresholds learned from held-out validation performance.
4. Explore online filtering: continuously update gating thresholds as the model improves.

6 Conclusion

Dmapo shows that multi-agent consensus filtering can reduce a 54k-candidate pool to 1,871 high-quality examples that outperform 5–10× larger baseline datasets on all four benchmarks—and produce the only model that improves over the pretrained base on MT-Bench.

The practical implication is clear: before scaling preference data collection, filter what already exists. The broader lesson is that in preference optimization, data quality dominates data quantity—contradictory labels are not merely wasteful but actively harmful, and removing them yields disproportionate gains.

References

Azar, M. G., Rowland, M., Piot, B., Guo, D., Calandriello, D., Valko, M., & Munos, R. (2023). A general theoretical paradigm to understand learning from human preferences. arXiv preprint arXiv:2310.12036.

Bengio, Y., Louradour, J., Collobert, R., & Weston, J. (2009). Curriculum learning. Proceedings of the 26th International Conference on Machine Learning, pp. 41–48.

Chen, L., Li, S., Yan, J., Wang, H., Gunaratna, K., Yadav, V., Tang, Z., Srinivasan, V., Zhou, T., Huang, H., & Jin, H. (2024). AlpaGasus: Training a better Alpaca with fewer data. International Conference on Learning Representations (ICLR).

Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., … & Zaremba, W. (2021). Training verifiers to solve math word problems. arXiv preprint arXiv:2110.14168.

Cui, G., Yuan, L., Ding, N., Yao, G., Zhu, W., Ni, Y., … & Sun, M. (2023). UltraFeedback: Boosting language models with high-quality feedback. arXiv preprint arXiv:2310.01852.

Ding, N., Chen, Y., Xu, B., Qin, Y., Zheng, Z., Hu, S., … & Sun, M. (2023). Enhancing chat language models by scaling high-quality instructional conversations. arXiv preprint arXiv:2305.14233.

Ethayarajh, K., Xu, W., Muennighoff, N., Jurafsky, D., & Kiela, D. (2024). KTO: Model alignment as prospect theoretic optimization. arXiv preprint arXiv:2402.01306.

Hong, J., Lee, N., & Thorne, J. (2024). ORPO: Monolithic preference optimization without reference model. Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP).

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., … & Chen, W. (2022). LoRA: Low-rank adaptation of large language models. International Conference on Learning Representations (ICLR).

Irving, G., Christiano, P., & Leike, J. (2018). AI safety via debate. arXiv preprint arXiv:1805.06949.

Jiang, A. Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D. S., Casas, D. d.l., … & Sayed, W. E. (2023). Mistral 7B. arXiv preprint arXiv:2310.06825.

Li, X., Zhang, T., Dubois, Y., Roig, G., Ranalli, A., Grangier, D., … & Malherbe, C. (2023). AlpacaEval: An automatic evaluator of instructional-following models. GitHub repository.

Li, G., Hammoud, H. A. A. K., Itani, H., Khizbullin, D., & Ghanem, B. (2023). CAMEL: Communicative agents for "mind" exploration of large language model society. Advances in Neural Information Processing Systems, 36.

Liu, W., Zeng, W., He, K., Jiang, Y., & He, J. (2024). What makes good data for alignment? A comprehensive study of automatic data selection in instruction tuning. International Conference on Learning Representations (ICLR).

Meng, Y., Xia, M., & Chen, D. (2024). SimPO: Simple preference optimization with a reference-free reward. Advances in Neural Information Processing Systems (NeurIPS).

Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., … & Leike, J. (2022). Training language models to follow instructions with human feedback. Advances in Neural Information Processing Systems, 35, 27730–27744.

Rafailov, R., Sharma, A., Mitchell, E., Manning, C. D., Ermon, S., & Newman, B. (2023). Direct preference optimization: Your language model is secretly a reward model. Advances in Neural Information Processing Systems, 36, 21068–21084.

Settles, B. (2009). Active learning literature survey. Computer Sciences Technical Report 1648, University of Wisconsin-Madison.

Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., … & Zhou, D. (2023). Self-consistency improves chain of thought reasoning in language models. International Conference on Learning Representations (ICLR).

Wang, Z., Dong, Y., Delalleau, O., Zeng, J., Shen, G., Egert, D., Zhang, J. J., Sreedhar, M. N., & Kuchaiev, O. (2024). HelpSteer2: Open-source dataset for training top-performing reward models. arXiv preprint arXiv:2406.08673.

Zha, D., Bhat, Z. P., Lai, K. H., Yang, F., Jiang, Z., Zhong, S., & Hu, X. (2023). Data-centric artificial intelligence: A survey. ACM Computing Surveys, 56(4), 1–47.

Zheng, L., Chiang, W. L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., … & Xing, E. P. (2024). Judging LLM-as-a-judge with MT-Bench and chatbot arena. Advances in Neural Information Processing Systems, 36, 46595–46623.

Zhou, J., Pasunuru, R., Mishra, S., Wang, T., Zhang, L., Dozat, T., … & Pasupat, P. (2023). Instruction-following evaluation for large language models. arXiv preprint arXiv:2310.07641.

Zhou, C., Liu, P., Xu, P., Iyer, S., Sun, J., Mao, Y., Ma, X., Efrat, A., Yu, P., Yu, L., Zhang, S., Ghosh, G., Lewis, M., Zettlemoyer, L., & Levy, O. (2023). LIMA: Less is more for alignment. Advances in Neural Information Processing Systems, 36.

A Judge Prompts and Configurations

Helpfulness Judge System Prompt.

> You are an expert evaluator assessing the HELPFULNESS of AI responses. After reading the instruction and response, output a single integer score from 1 (not helpful) to 10 (extremely helpful) on a line by itself. Optionally add one sentence of reasoning after the score.

Factuality Judge System Prompt.

> You are an expert evaluator assessing the FACTUAL ACCURACY of AI responses. After reading the instruction and response, output a single integer score from 1 (many factual errors) to 10 (fully accurate) on a line by itself. Optionally add one sentence of reasoning after the score.

Conciseness Judge System Prompt.

> You are an expert evaluator assessing the CONCISENESS of AI responses. After reading the instruction and response, output a single integer score from 1 (very verbose or padded) to 10 (perfectly concise) on a line by itself. Optionally add one sentence of reasoning after the score.

Process Critic System Prompt.

> You are a reasoning quality evaluator. Read the instruction and response carefully. Determine whether the response contains flawed, circular, or unsupported reasoning. Reply on the first line with YES or NO. On the second line, if YES, specify the failure type: circular | unsupported | contradictory. On the third line, give one short reason.

B Training Hyperparameters

B.1 LoRA Configuration

- Rank (r): 16
- Alpha (α): 32
- Dropout: 0.05
- Bias: none
- Target modules: q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj
- Task type: CAUSAL_LM

B.2 Training Arguments

- Optimizer: AdamW (fused kernels)
- Learning rate: 5.0×10^{-5}
- LR scheduler: cosine with 5% warmup
- Per-device batch size: 2
- Gradient accumulation steps: 8
- Effective batch size: 16
- Num epochs: 1
- Max token length: 1024
- Precision: bfloat16
- Save frequency: every 200 steps
- Eval frequency: every 200 steps

C Confidence Gate Thresholds

- Variance threshold (τ_{var}): 2.5
- Desirable threshold (τ_{des}): 7.0
- Undesirable threshold (τ_{und}): 4.0
- Critic penalty weight (α): 0.15

These thresholds were selected to optimize the balance between data quality and quantity, as shown in Table 4.

D Reproducibility Checklist

- Code availability: We will release all code, configurations, and trained model checkpoints on GitHub.
- Dataset: We use publicly available datasets (UltraFeedback, HelpSteer2) and will release the gated dataset.
- Random seeds: All experiments use a fixed seed (42) for reproducibility.

- Computational requirements: 8× RTX PRO 6000 Blackwell GPUs (∼760 GB total VRAM). Generation throughput: ∼2,000 candidates/GPU-hour.
- Estimated runtime: Generation + scoring: 4–6 hours; training per model: 30 min–2 hours; evaluation: 2–4 hours.
- Software versions: PyTorch 2.1, TRL 0.29.0, PEFT 0.18.1, Transformers 4.36.

E Ethics and Broader Impact

Dmapo uses LLM judges to filter training data, which introduces a risk of amplifying biases present in the judge models. If judges systematically favor certain response styles or viewpoints, the filtered dataset will inherit these biases, potentially narrowing the diversity of the fine-tuned model's outputs. We mitigate this by using three judges with orthogonal evaluation dimensions and by requiring consensus, which reduces the influence of any single judge's bias. However, practitioners should audit judge behavior on domain-specific data before deployment.

All datasets used (UltraFeedback, HelpSteer2) are publicly available and do not contain personally identifiable information. We will release our code, configurations, and the gated dataset to support reproducibility and enable further research on data-quality-driven alignment.